

实验五 分类算法（2）

（一）目的和要求

- （1）掌握分类算法中的集成学习的概念的使用场景
- （2）掌握使用集成学习的方法和步骤
- （3）掌握模型参数优化的方法

（二）实验知识准备

集成学习：集成学习是一种通过结合多个基学习器来提升模型性能的方法。其基本思想是利用多个模型的优势，减少单一模型的偏差和方差，提高预测的准确性。集成学习主要分为两类：**Bagging** 和 **Boosting**。**Bagging** 通过对训练数据进行重采样，构建多个独立的模型，然后将它们的预测结果进行平均（回归）或投票（分类）；而 **Boosting** 则通过串行训练模型，每个模型都试图纠正前一个模型的错误，最终综合各模型的预测结果。

（1）**随机森林：**随机森林是 **Bagging** 的一个具体实现，使用多棵决策树作为基学习器。其基本原理是从全数据集中随机选择样本和特征，构建多棵树，每棵树独立地进行分类或回归。最终的预测结果通过对所有树的结果进行投票（分类）或平均（回归）得出，随机森林有效地减少了过拟合，并提高了模型的稳定性和准确性。

（2）**模梯度提升决策树（GBDT）：**GBDT 是一种基于 **Boosting** 思想的集成学习方法，通过逐步构建决策树模型来优化目标损失函数。每棵新树的构建都是在前一棵树的基础上，通过最小化残差来改进模型。**GBDT** 在处理非线性关系和特征交互方面表现出色，是许多机器学习竞赛中的常用模型。

（3）**XGBoost：**XGBoost 是 GBDT 的一种高效实现，具有更快的训练速度和更好的性能。它采用了正则化技术，以减少过拟合，并支持并行计算，从而加快模型训练过程。**XGBoost** 还具有自动处理缺失值的能力，并且在处理大规模数据集时表现优异，是许多实际应用和竞赛中的热门选择。

（4）**LightGBM：**LightGBM 是微软开发的一种高效的 GBDT 实现，针对大规模数据集进行了优化。它采用了基于直方图的决策树算法，能够显著提高训练速度和内存效率。**LightGBM** 支持类别特征的直接处理，且通过调节参数可

以有效控制过拟合，适合于大数据场景和高维特征。

■ 参数优化

1. 网格搜索：网格搜索是一种穷举法，通过在预定义的参数空间中列出所有可能的参数组合，逐一训练模型并评估其性能。该方法简单易懂，但在参数空间较大时，计算开销会显著增加。
2. 随机搜索：随机搜索通过随机选择参数组合进行模型训练，相较于网格搜索，它能更有效地探索参数空间，尤其是在高维空间中。随机搜索通常能在较短时间内找到较优的参数组合。
3. 贝叶斯优化：贝叶斯优化是一种基于概率模型的优化方法，通过构建目标函数的概率模型，逐步更新模型以找到最佳参数组合。该方法能够有效减少评估次数，适合于计算成本高的模型训练。
4. 遗传算法：遗传算法模拟自然选择过程，通过选择、交叉和变异等操作在参数空间中进行搜索。这种方法适合于复杂的优化问题，能够在较大范围内寻找全局最优解。
5. 超参数优化工具：借助一些现成的库和工具，如 Optuna、Hyperopt 和 Ray Tune 等，可以方便地进行超参数优化。这些工具集成了多种优化算法，能够自动化参数搜索过程，并提供可视化和分析功能。

■ 模型解释

1. 特征重要性分析：通过评估各特征对模型预测结果的影响程度，特征重要性分析可以帮助识别哪些特征在模型决策中起到关键作用。常用的方法包括基于树模型的特征重要性（如随机森林和 XGBoost）和基于线性模型的系数分析。
2. SHAP：SHAP 值基于合作博弈论，通过计算特征对模型预测的贡献来提供解释。SHAP 不仅能够量化每个特征对单个预测的影响，还能通过全局解释分析特征的重要性的模型的整体行为。
3. 部分依赖图：部分依赖图用于可视化单个或多个特征对模型预测的影响。通过绘制特征值的变化与预测值之间的关系，PDP 能够揭示特征与预测结果之间的非线性关系。
4. 对抗性样本分析：通过生成对抗性样本并观察模型的预测变化，可以深入

了解模型的决策边界和脆弱性。这种方法有助于识别模型在特定条件下的局限性。

5. 局部可解释模型-agnostic 解释：LIME 是一种模型无关的解释方法，通过在目标样本附近生成局部线性模型，来近似复杂模型的行为。LIME 能够提供单个预测的可解释性，帮助用户理解模型在特定输入下的决策原因。

（三）实验内容及步骤

（1）使用集成学习方法识别葡萄酒类别

葡萄酒识别数据集相关介绍见实验四。

1. 步骤一，导入本项目所需要的模块和包，如图 1 所示。

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 import pandas as pd
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
6 from sklearn.datasets import load_wine
7 from sklearn.ensemble import RandomForestClassifier
8 from sklearn.model_selection import GridSearchCV
9 from sklearn.feature_selection import SelectFromModel
10 from sklearn.metrics import fl_score
11 import numpy as np
```

图 1 导入所需模块

2. 步骤二，使用 `load_data()` 函数从 Keras 库中导入葡萄酒数据集到本地计算机中。参考代码如图 2 所示。

```
1 # 导入数据，分别为输入特征和标签
2 wine = load_wine()
3 X=wine.data
4 Y=wine.target
5 # 划分训练集和测试集
6 X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2,
    random_state=42)
7 # 输出结果
8 print("训练集特征形状:", X_train.shape)
9 print("测试集特征形状:", X_test.shape)
10 print("训练集标签形状:", y_train.shape)
11 print("测试集标签形状:", y_test.shape)
```

图 2 从 Keras 加载数据集

3. 步骤三，定义随机森林分类器并行参数优化，主要代码如图 3 所示。

```
1 #模型训练和参数优化
2 rf = RandomForestClassifier(random_state=42)
3 # 定义参数网格
4 param_grid = {
5     'n_estimators': [50, 100, 200],
6     'max_depth': [None, 5, 10],
7     'min_samples_split': [2, 5, 10]
8 }
9 # 执行网格搜索
10 grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, scoring='accuracy', cv=3)
11 grid_search.fit(X_train, y_train)
12 # 输出最佳参数
13 best_params = grid_search.best_params_
14 print("最佳参数:", best_params)
```

图 3 定义并优化分类器

4. 步骤四，使用最佳参数训练模型，示例代码如图 4 所示。

```
1 # 使用最佳参数训练模型
2 best_model = grid_search.best_estimator_
3 best_model.fit(X_train, y_train)
```

图 4 定义并训练分类器

打印出分类器的混淆矩阵，如图 5 所示。

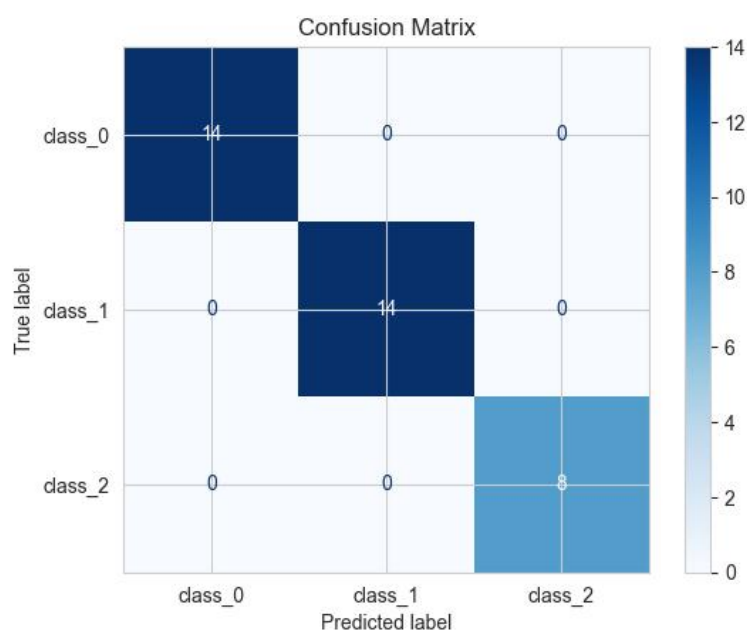


图 5 分类器混淆矩阵

5. 步骤五，获取特征重要性排序并进行可视化，示例代码如下图 6。

```
1 # 特征重要性排序
2 # 获取特征重要性
3 importances = best_model.feature_importances_
4 feature_names = X.columns
5 # 创建数据框以便排序
6 feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
7 feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)
8 # 可视化特征重要性
9 plt.figure(figsize=(10, 6))
10 sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
11 plt.title('Feature Importance from Random Forest')
12 plt.xlabel('Importance')
13 plt.ylabel('Feature')
14 plt.show()
```

图 6 特征重要性排序

特征重要性排序图，如图 7 所示。

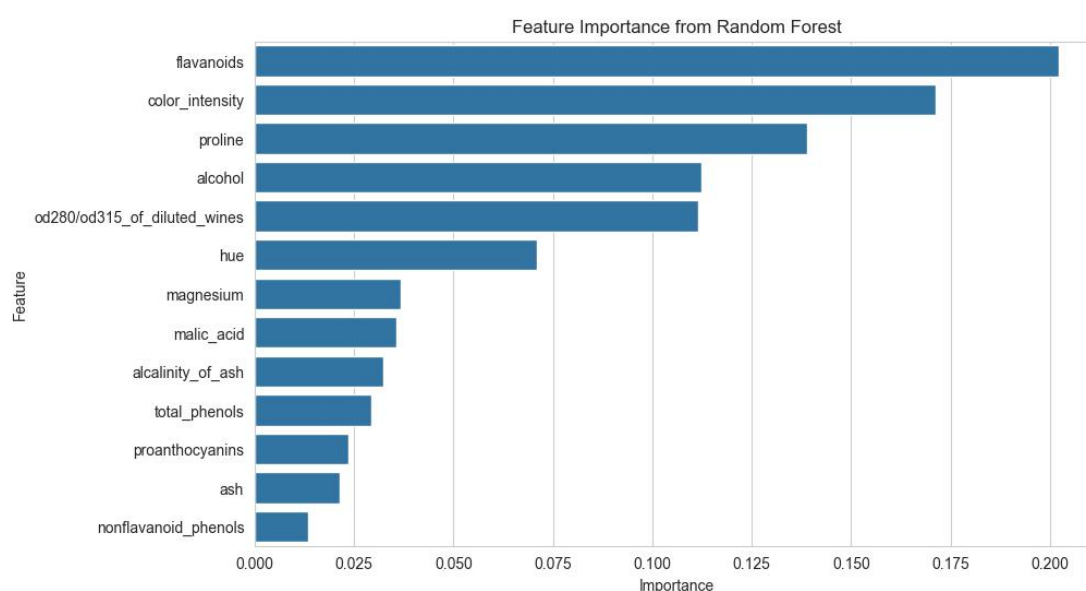


图 7 重要性排序可视化

6. 步骤六，进行特征选择，示例代码如下图 8 所示。

```
1 selector = SelectFromModel(best_model, threshold="mean", prefit=True)
2 X_train_selected = selector.transform(X_train)
3 X_test_selected = selector.transform(X_test)
```

图 8 进行特征选择

7. 步骤七，对选择后的模型，输出分类结果的评价指标，示例代码如下图 9 所示。

```

1 # 对选择后的模型训练和评估
2 rf_selected = RandomForestClassifier(random_state=42)
3 rf_selected.fit(X_train_selected, y_train)
4 y_pred_selected = rf_selected.predict(X_test_selected)
5 # 计算选择后的性能指标
6 accuracy_selected = accuracy_score(y_test, y_pred_selected)
7 f1_selected = f1_score(y_test, y_pred_selected, average='weighted')

```

图 9 输出选择后模型的评价指标

8. 步骤八，绘制性能对比图，示例代码如图 10 所示。

```

1 # 绘制性能对比图
2 performance_metrics = ['Accuracy', 'F1 Score']
3 original_performance = [accuracy_original, f1_original]
4 selected_performance = [accuracy_selected, f1_selected]
5 x = np.arange(len(performance_metrics)) # x轴位置
6 # 创建条形图
7 width = 0.35 # 条的宽度
8 fig, ax = plt.subplots(figsize=(10, 6))
9 bars1 = ax.bar(x - width/2, original_performance, width, label='Original Features')
10 bars2 = ax.bar(x + width/2, selected_performance, width, label='Selected Features')
11 # 添加标签和标题
12 ax.set_ylabel('Scores')
13 ax.set_title('Classifier Performance Comparison Before and After Feature Selection')
14 ax.set_xticks(x)
15 ax.set_xticklabels(performance_metrics)
16 ax.legend()
17 # 显示数值在条上
18 def add_value_labels(bars):
19     for bar in bars:
20         yval = bar.get_height()
21         ax.text(bar.get_x() + bar.get_width()/2, yval, round(yval, 2), ha='center', va='bottom')
22
23 add_value_labels(bars1)
24 add_value_labels(bars2)
25 plt.tight_layout()
26 plt.show()

```

图 10 绘制性能对比图

对比结果如图 11 所示，

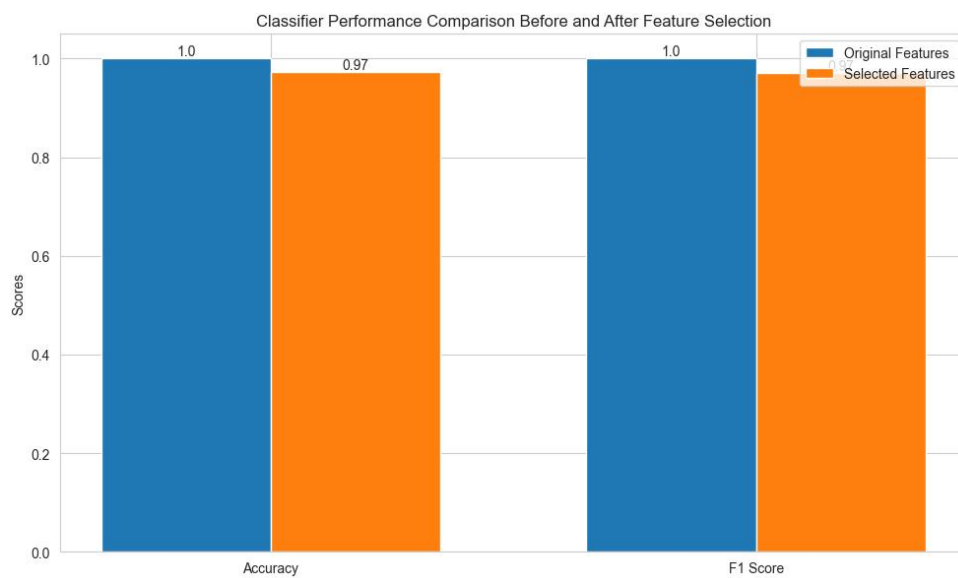


图 11 性能比对图

（2）实训项目

导入 `scikit-learn` 库中的鸢尾花数据集或其他数据集，任选两种不同的集成学习方法（`XGBOOST`，`GBDT`，`LIGHTGBM`）对选定进行分类。鸢尾花数据集各列说明如表 2 所示。完成以下任务：

1. 步骤一，导入需要的库文件及数据集。
2. 步骤二，对数据进行初步探索，包括查看数据的基本信息和可视化特征之间的关系。
3. 步骤三，将数据集分为训练集和测试集，通常按 8:2 或 7:3 的比例划分。
4. 步骤四，对选定的分类方法进行参数优化。
5. 步骤五，对选定的分类器进行特征选择，选出最优特征。
6. 步骤五，使用测试集进行预测，并将结果保存下来。
7. 步骤六，分别比较参数优化前后及特征选择前后两个不同分类器的性能。
8. 步骤七，对比较结果进行可视化。